

Análisis de Algoritmos de Backtracking

Autor: Dylan Rojas

Fecha: 13-11-2025

Objetivo: Analizar dos implementaciones de un algoritmo de *backtracking* cuyo objetivo es encontrar un camino en una cuadrícula de $N \times N$ que visite cada casilla exactamente una vez.

Introducción

Este informe analiza dos implementaciones de un algoritmo de **backtracking (vuelta atrás)** en Python. El objetivo de ambos scripts es idéntico: encontrar un camino en una cuadrícula (tablero) de $N \times N$ que visite cada casilla exactamente una vez.

El camino debe comenzar en la casilla $(0, 0)$ y terminar en la casilla $(N-1, N-1)$. El número de pasos debe ser exactamente $N * N$ para asegurar que todas las casillas han sido visitadas.

Los dos códigos resuelven el mismo problema pero utilizan dos enfoques de implementación fundamentalmente diferentes:

1. **Código 1 (Iterativo):** Utiliza un bucle **while** y una estructura de datos de Pila (Stack) explícita para gestionar el estado de la búsqueda.
2. **Código 2 (Recursivo):** Utiliza la propia pila de llamadas del sistema (call stack) de forma implícita, mediante una función que se llama a sí misma.

Análisis del Código 1: Enfoque Iterativo

Código:

Numero de línea	Código
1	def mostrar_tablero(T):
2	"""Muestra el resultado final de forma clara."""
3	for fila in T:
4	print(" ".join(f"{c:2}" for c in fila))
5	
6	def buscar_ruta_simple(N=3):
7	"""Busca la ruta en un tablero de 3x3 de forma iterativa."""
8	
9	Tablero = [[0] * N for _ in range(N)] # Tablero vacío
10	Pila = [] # Lista de pasos dados (la Pila)
11	
12	# Empezamos: Fila 0, Columna 0, Paso 1
13	f, c, paso = 0, 0, 1
14	Tablero[f][c] = paso
15	Pila.append((f, c, paso))
16	
17	# Direcciones: (f±1, c±0) o (f±0, c±1)
18	MOVIMIENTOS = [(1, 0), (0, 1), (-1, 0), (0, -1)]
19	
20	while Pila:
21	# Mira dónde estamos ahora (el último paso)
22	f, c, paso = Pila[-1]
23	
24	# 1. ¡Ganamos! Si llegamos al final (2, 2) y dimos 9 pasos (3*3)
25	if f == N - 1 and c == N - 1 and paso == N * N:
26	return Tablero
27	
28	# 2. Intentamos movernos a una casilla vecina LIBRE
29	encontrado_siguiente = False
30	
31	for df, dc in MOVIMIENTOS:
32	nf, nc = f + df, c + dc # Nueva posición
33	npaso = paso + 1 # Siguiente número de paso
34	
35	# Chequeo rápido: ¿Dentro del tablero y con un 0 (vacío)?
36	if (0 <= nf < N and 0 <= nc < N and Tablero[nf][nc] == 0):
37	# ¡Avanzamos! Marcamos la casilla y la añadimos a la
	Pila
38	Tablero[nf][nc] = npaso
39	Pila.append((nf, nc, npaso))
40	encontrado_siguiente = True

41	Break
42	
43	# 3. Retroceso (Backtracking)
44	if not encontrado_siguiente:
45	# Si no hay dónde ir, quitamos el último paso de la Pila...
46	Pila.pop()
47	# ... y borramos la marca de esa casilla (la ponemos a 0)
48	if Tablero[f][c] != 0:
49	Tablero[f][c] = 0
50	
51	return None # No se encontró la ruta

Análisis de la Metodología

Este script simula manualmente el proceso de recursión utilizando un bucle **while** y una Pila (**Pila = []**).

- **Gestión de Estado:** La **Pila** es la estructura de datos central. Almacena tuplas (**f, c, paso**) que representan el camino actual. El estado actual *siempre* es el elemento en la cima de la pila (**Pila[-1]**).
- **Lógica de Avance (Exploración):**
 1. El bucle **while Pila:** se ejecuta mientras haya caminos por explorar.
 2. Se "espía" (mira sin sacar) el último paso en **Pila[-1]**.
 3. Se comprueba si es la solución.
 4. Se itera sobre los **MOVIMIENTOS** para encontrar una casilla vecina válida (dentro de límites y vacía **== 0**).
 5. Si se encuentra una, se marca el **Tablero**, se añade el nuevo paso (**nf, nc, npaso**) a la Pila (un **"push"**), y se usa **break** para reiniciar el bucle **while** desde esta *nueva* posición.
- **Lógica de Retroceso (Backtracking):**
 1. Si el bucle **for df, dc in MOVIMIENTOS:** termina **sin** encontrar un siguiente paso (la bandera **encontrado_siguiente** es **False**), significa que la posición actual (**f, c**) es un callejón sin salida.
 2. Se ejecuta el **backtrack**:
 - **Pila.pop()**: Se elimina el callejón sin salida de la Pila, retrocediendo al estado anterior.
 - **Tablero[f][c] = 0**: Se "limpia" la casilla en el tablero, marcándola como no visitada para que otros caminos puedan usarla.
 3. El bucle **while** continúa, pero ahora desde la posición que quedó en la cima de la Pila.
- **Finalización:** Si la **Pila** se vacía, significa que se ha retrocedido hasta el inicio y no quedan más opciones. Se devuelve **None**.

Análisis del Código 2: Enfoque Recursivo

Código Fuente:

Numero de línea	Código
1	def mostrar(T):
2	""Muestra el resultado final.""
3	for fila in T:
4	print(" ".join(f"{c:2}" for c in fila))
5	
6	def jugar(T, x, y, p):
7	""Busca la ruta con Ensayo y Error.""
8	N = len(T)
9	
10	# 1. ¿Ganamos? (Llegar al final con todos los pasos hechos)
11	if x == N-1 and y == N-1 and p == N*N:
12	T[x][y] = p
13	return True
14	
15	# 2. ¿La casilla es válida? (Dentro y Vacía)
16	if 0 <= x < N and 0 <= y < N and T[x][y] == 0:
17	T[x][y] = p # Dejar una marca (el número de paso)
18	
19	# 3. Probar Abajo, Derecha, Arriba, Izquierda. Si una ruta funciona, ¡listo!
20	if (jugar(T, x+1, y, p+1) or
21	jugar(T, x, y+1, p+1) or
22	jugar(T, x-1, y, p+1) or
23	jugar(T, x, y-1, p+1)):
24	return True
25	
26	# 4. Si fallan las 4: Borrar la marca y retroceder
27	T[x][y] = 0
28	return False
29	
30	# ... (Bloque de ejecución omitido para el análisis) ...

Análisis de la Metodología:

Este script utiliza la recursión, un enfoque más elegante y directo para problemas de backtracking. La gestión de la "Pila" es implícita, manejada por la Pila de Llamadas (Call Stack) de Python.

- **Gestión de Estado:** El estado no se almacena en una estructura de datos explícita, sino en los argumentos de la función `jugar(T, x, y, p)`. Cada llamada recursiva representa un paso en el camino. El Tablero (`T`) se modifica "in-place" (se pasa por referencia).
- **Casos Base (Condiciones de Parada):**
 1. **Caso Base de Éxito:** Si se alcanza la casilla final (`N-1, N-1`) y se han dado todos los pasos (`p == N*N`), se marca la casilla y se devuelve `True`. Este `True` se "propaga" hacia atrás en la cadena de llamadas, finalizando el algoritmo.
 2. **Caso Base de Fallo:** La comprobación `if 0 <= x < N and 0 <= y < N and T[x][y] == 0`: actúa como el principal caso de fallo. Si la llamada a `jugar` se hace con coordenadas fuera del tablero, o en una casilla ya visitada (`T[x][y] != 0`), la función no entra al bloque `if` y termina, devolviendo `None` (que en el `if` de la llamada anterior se evalúa como `False`).
- **Lógica de "Choose, Explore, Un-choose":** Este patrón es la firma del backtracking recursivo.
 1. **"Choose" (Elegir):** `T[x][y] = p`. Se "toma una decisión" y se marca la casilla actual.
 2. **"Explore" (Explorar):** `if (jugar(...) or jugar(...) ...)`: Se llama recursivamente a la función para todas las direcciones posibles. Gracias al "short-circuit" del `or`, si la primera llamada (Abajo) eventualmente devuelve `True`, las otras 3 direcciones no se exploran.
 3. **"Un-choose" (Retroceder):** Si el bloque `if` de exploración falla (es decir, las 4 direcciones devolvieron `False`), se ejecuta el código de backtracking:
 - `T[x][y] = 0`: Se deshace la decisión ("se borra la marca").
 - `return False`: Se informa a la llamada anterior que este camino fue un callejón sin salida.

Comparativa

Ambos algoritmos son conceptualmente idénticos: implementan una **Búsqueda en Profundidad (DFS)** con retroceso. La diferencia radica puramente en la implementación.

Característica	Código 1 (Iterativo con Pila)	Código 2 (Recursivo)
Gestión de Estado	Explícita. El programador gestiona una Pila = [] .	Implícita. El sistema gestiona la Pila de Llamadas.
Legibilidad	Menos legible. La lógica del bucle while , el break y la bandera encontrado_siguiente hacen el flujo más complejo de seguir.	Más legible. El patrón "Choose, Explore, Un-choose" es muy claro y se alinea directamente con la teoría del backtracking.
Complejidad	Más complejo de escribir. Requiere gestionar manualmente el avance y retroceso (push/pop) y la limpieza del tablero.	Más simple de escribir. La lógica de "retroceder" es una simple sentencia return y una limpieza.
Rendimiento	Potencialmente más rápido (marginamente) al evitar la sobrecarga de las llamadas a funciones.	La sobrecarga (overhead) de miles de llamadas a funciones puede ser ligeramente más lenta.
Límites de Memoria	Limitado solo por la memoria RAM del sistema. Puede manejar problemas muy "profundos".	Limitado por el límite de profundidad de recursión de Python (usualmente 1000). Fallará con un RecursionError si N es muy grande (ej. N=40).

Conclusión

El **enfoque recursivo (Código 2)** es generalmente preferido para problemas de backtracking de este tipo debido a su **claridad y elegancia**. El código es más corto, más fácil de entender y de razonar sobre él.

El **enfoque iterativo (Código 1)** es una técnica poderosa que se vuelve necesaria cuando:

1. La profundidad del problema es tan grande que superaría el límite de recursión (ej. un tablero de 50x50).
2. Se necesita un control más granular sobre la pila de exploración (algo poco común).

Para un problema de tamaño $N=3$ o $N=4$, como el planteado, ambas soluciones son perfectamente válidas, pero la versión recursiva es superior en términos de simplicidad de desarrollo.